Physics-Informed Solar Energy Analytics

Detailed Professor-Readiness, Technical Study, and Execution Guide

Project: Physics-Informed Solar Energy Forecasting and Fault Detection

Purpose: A learning and portfolio guide for a graduate applicant presenting an applied machine-learning project in scientific/energy analytics.

Important integrity statement: *This repository uses synthetic, simulated data. It is a reproducible educational portfolio project, not evidence of work on a real solar farm, not a validated commercial monitoring system, and not a field study. Explain this clearly to professors. Never describe simulated observations as measurements collected from a plant.*

1. What the project is about

Solar photovoltaic (PV) systems convert sunlight into electrical power. Their output changes with solar irradiance, module temperature, time of day, weather, and equipment condition. A useful analytics system can estimate expected power, forecast near-future output, and flag periods when measured output is unusually low.

This project demonstrates three connected ideas:

1. **Physical reasoning:** a simplified PV relationship estimates how irradiance and module temperature influence expected power.
2. **Forecasting:** machine-learning models use historical and environmental features to predict power one hour ahead.
3. **Fault detection:** a classifier attempts to distinguish normal operating periods from simulated abnormal/fault periods.

The central research question is:

Can a simple physics-informed baseline and machine-learning models provide useful next-hour solar-power forecasts and identify simulated underperformance, and how should their limitations be evaluated?

This is a portfolio demonstration of a workflow. It does not prove that the model will perform equally well on a real plant.

2. Why this is relevant to graduate research

The project connects physics, computing, statistics, and energy systems. It can support conversations with faculty whose work involves machine learning, scientific computing, time-series analysis, anomaly detection, renewable energy, optimisation, or data-driven physical systems.

It demonstrates that you can:

- formulate a measurable problem;
- understand the physical meaning of input variables;
- inspect and prepare a dataset;
- separate training and test periods chronologically;
- establish simple baselines;
- train and evaluate machine-learning models;

- interpret metrics and visualisations;
- recognise class imbalance, leakage, synthetic-data limitations, and overclaiming;
- organise code, data, notebooks, tests, and documentation for reproducibility.

Professors are not merely looking for a high accuracy score. They want to see sound reasoning, honest reporting, evidence that you understand your code, and the ability to explain what you would improve.

3. Dataset: what it contains and what it does not

The supplied CSV contains **17,520 hourly records** generated synthetically for the years 2024–2025. The generator creates plausible patterns for irradiance, module temperature, time, and PV output, and injects simulated abnormal intervals for the fault-detection exercise.

The data are not downloaded from a real PV installation. They are not measurements from KNUST, an employer, a named solar farm, or a research partner.

Before modelling, inspect:

- number of rows and columns;
- column names and data types;
- timestamp range and frequency;
- missing values and duplicate timestamps;
- impossible or suspicious values;
- daylight versus night-time records;
- target distribution and fault prevalence.

Useful commands:

```
import pandas as pd

df = pd.read_csv("data/synthetic_solar_pv_hourly.csv")
print(df.shape)
print(df.head())
print(df.info())
print(df.isna().sum())
print(df.describe(include="all"))
print(df.columns.tolist())
```

Questions to answer:

- Are timestamps parsed correctly?
- Are records hourly and in chronological order?
- Are night-time power values close to zero?
- Are there negative irradiance or impossible temperature values?
- What fraction of observations are labelled as faults?
- How were faults injected by the data generator?

A professor may ask why synthetic data were used. A good answer is that synthetic data allow a transparent, reproducible first experiment without misrepresenting private or unavailable plant data. The next step would be validation on an open, independently sourced PV dataset or authorised plant measurements.

4. Physics-informed component

A simplified expected-power expression used in the project is:

$$
P_{expected}=P_{rated}\left(\frac{G}{1000}\right)
\left[1+\gamma(T_{module}-25)\right]
$$

where:

- $P_{expected}$: simplified expected PV power in kW;
- P_{rated}: nominal rated system capacity (100 kW in this demonstration);
- G: plane-of-array irradiance in W/m²;
- 1000 W/m²: reference irradiance;
- γ: assumed temperature coefficient (–0.004 per °C in this demonstration);
- T_{module}: module temperature in °C;
- 25°C: reference cell/module temperature in the simplified relationship.

Interpretation:

- Higher irradiance generally increases available PV power.
- Higher module temperature generally reduces PV module efficiency when the temperature coefficient is negative.
- The equation is a simplified teaching approximation.

Limitations: It omits many real-world effects, including inverter efficiency and clipping, shading, soiling, module mismatch, wiring losses, degradation, angle-of-incidence effects, spectral effects, detailed thermal dynamics, and manufacturer-specific behaviour. Do not call it a complete PV performance model or engineering design equation.

Be prepared to explain the sign of γ: because it is negative, increasing module temperature above the reference temperature reduces the temperature adjustment factor.

5. Forecasting task

The forecasting task predicts PV power one hour into the future. The code uses a chronological train/test split (80% earlier observations for training and 20% later observations for testing).

Why chronological splitting matters:

- Time-series observations are ordered.
- A random split can mix future and past records.
- A chronological split better represents the practical situation of training on the past and forecasting a later period.

The project compares a simple baseline with a Random Forest model. The baseline is important: a sophisticated model is only useful if it improves on a reasonable simple alternative.

Reported results from the generated run:

Model	MAE (kW)	RMSE (kW)	R²
Mean baseline	19.8769	22.7254	–0.000421

| Random Forest | 3.6583 | 5.9076 | 0.932394 |

Interpretation:

- **MAE** is the average absolute prediction error. Lower is better.
- **RMSE** penalises large errors more strongly than MAE. Lower is better.
- **R²** compares the model with predicting the test-set mean. A value near 1 indicates strong fit relative to that baseline; a negative value means worse than the mean predictor.

The Random Forest scores are strong on this synthetic test split. They must not be presented as expected real-world performance. The synthetic data generator may create regular patterns that are easier for a model to learn than real weather and plant behaviour.

Potential improvements:

- compare against persistence (next-hour power equals current-hour power);
- compare against a physics-only estimate;
- use rolling-origin or expanding-window validation;
- report performance by season, daylight level, and weather regime;
- include prediction intervals or uncertainty estimates;
- test on a separate real-world dataset;
- evaluate generalisation to a different site.

6. Fault detection task

The fault task is a supervised classification exercise. It uses simulated fault labels and attempts to classify operating records as normal or faulty.

Reported results from the generated run:

| Metric | Majority baseline | Random Forest |

| --- | ---: | ---: |

| Accuracy | 0.971787 | 0.992476 |

| Precision | 0 | 1.000000 |

| Recall | 0 | 0.733333 |

| F1 | 0 | 0.846154 |

| ROC-AUC | Not meaningful / unavailable | 0.985455 |

The high majority-baseline accuracy is a warning: faults are rare, so a model that predicts “normal” for every record can appear accurate while detecting no faults.

Definitions:

- **Accuracy:** proportion of all predictions that are correct.
- **Precision:** among records predicted faulty, the proportion that are truly faulty.
- **Recall:** among all truly faulty records, the proportion detected.
- **F1:** harmonic mean of precision and recall.
- **ROC-AUC:** ranking/discrimination measure across classification thresholds; it does not by itself choose a practical alarm threshold.

The Random Forest has high precision in this run and detects about 73.3% of simulated faults. This still means some faults are missed. In a real plant, the cost of false alarms and missed faults must be considered.

Important leakage/shortcut risk: the feature set includes variables such as `performance_ratio` and `actual_power_kw`, which may be closely connected to how the synthetic fault labels were generated. A classifier may learn the injected labelling mechanism rather than discover general fault signatures. The README recommends an ablation experiment excluding these variables. Do that before claiming robust fault detection.

Recommended ablation:

1. Reproduce the original result.
2. Remove `performance_ratio` and `actual_power_kw` from the classifier inputs.
3. Retrain using the same split and evaluation metrics.
4. Compare precision, recall, F1, ROC-AUC, and the confusion matrix.
5. Explain why performance changes.
6. If possible, build a fault-detection model using only variables available before the fault is confirmed.

7. How to interpret the reports and figures

Open the `reports/` folder and understand each output rather than memorising its values.

- `forecast_metrics.csv`: numerical forecast comparison. Explain each metric and identify the better result.
- `fault_detection_metrics.csv`: classification comparison. Explain why accuracy alone is misleading.
- `fault_classification_report.txt`: class-wise precision, recall, and F1.
- `fault_confusion_matrix.csv`: counts of true/false positives and negatives.
- `fault_permutation_importance.csv`: estimates how much model score changes when a feature is shuffled. It is a model-specific diagnostic, not proof of causation.
- `run_summary.json`: machine-readable summary of the analysis run.
- forecast figure: inspect whether predicted and actual curves follow the same broad patterns and where errors are largest.
- confusion-matrix figure: identify missed faults and false alarms.
- permutation-importance figure: identify influential model inputs, then consider whether they could encode the target construction.

Do not say that feature importance proves a physical variable causes power loss. Predictive association is not causation.

8. What a professor may expect you to explain

You should be able to answer these questions in your own words:

1. What real scientific or engineering problem motivates the project?
2. Why is solar PV output variable?
3. What does irradiance represent, and why does it matter?
4. Why does module temperature affect output?
5. What does the simplified physics equation assume?
6. Why did you choose a one-hour-ahead forecasting horizon?
7. Why did you split the data chronologically?

8. What would go wrong with a random train/test split for time series?
9. What is the difference between MAE and RMSE?
10. What does a negative or near-zero R^2 mean?
11. Why is a mean baseline insufficient, and what other baselines should be tested?
12. Why can accuracy be misleading for rare faults?
13. What do precision and recall mean in an operational alarm system?
14. What is a false positive? What is a false negative?
15. What is data leakage, and how might it occur here?
16. Why should `performance_ratio` and `actual_power_kw` be removed in an ablation test?
17. What does permutation importance tell you, and what does it not tell you?
18. Why is synthetic data a limitation?
19. What evidence would be required before deployment?
20. How would you test the model on another solar site?
21. How would you handle missing sensor readings and irregular sampling?
22. How would you quantify uncertainty in the forecast?
23. How would you choose an alarm threshold based on maintenance costs?
24. What is your individual contribution, and which parts did you implement and verify?
25. What would you do next if given a research assistantship and access to real PV data?

A strong answer is specific, honest, and tied to code or results. If you do not know an answer, say what you would investigate rather than inventing one.

9. A credible research extension plan

A useful next-stage research plan could be:

Stage 1 — Reproducibility: run the repository from a clean environment, record package versions, and verify all reported metrics.

Stage 2 — Stronger baselines: compare persistence, moving-average, physics-only, and machine-learning forecasts.

Stage 3 — Leakage audit: perform the fault-feature ablation and document the effect.

Stage 4 — Robust validation: use rolling time-based validation and test across different seasons or sites.

Stage 5 — Uncertainty: add prediction intervals or quantile regression so forecasts communicate confidence, not just a point estimate.

Stage 6 — Explainability: use permutation importance or SHAP carefully, check stability across time periods, and avoid causal claims.

Stage 7 — External validation: evaluate on an openly available real PV dataset or data provided with permission. Document differences in sensors, site, climate, capacity, and data quality.

Stage 8 — Research question: for example, “How does combining physical constraints with time-series machine learning affect forecast accuracy and reliability under changing weather and equipment conditions?”

10. How to present this project to professors

Use a concise and accurate description:

I developed a reproducible, physics-informed solar energy analytics portfolio project using a two-year synthetic hourly PV dataset. I implemented a simplified irradiance- and temperature-based expected-power relationship, compared a Random Forest with a baseline for one-hour-ahead power forecasting, and evaluated a supervised model for simulated fault detection. I used chronological evaluation and examined metrics including MAE, RMSE, R^2, precision, recall, F1, and ROC-AUC. The project is explicitly synthetic, and I am extending it through feature-leakage ablation, stronger time-series baselines, and external validation.

Be prepared to share:

- the public repository URL;
- a short explanation of the physical assumptions;
- the notebook and source code;
- the output reports;
- one limitation you discovered;
- one concrete research extension that matches the professor's work.

Do not claim that a professor has endorsed the project or that it is published research. It is a self-directed portfolio project unless you have separately submitted it to a recognised venue.

11. Academic integrity and responsible claims

- Clearly label the dataset as synthetic in the repository, CV, SOP, and interviews.
- Do not claim field deployment, real plant monitoring, measured energy savings, or industrial validation.
- Do not claim a metric from a report unless you can reproduce it.
- Do not claim sole authorship of components you did not personally implement or understand.
- Identify assistance or tools appropriately if a university asks about AI-assisted development.
- Keep a record of the commands, package versions, changes, and experiments you personally run.
- Remove secrets, credentials, private data, and personal information from the repository.
- If you later use real data, check the dataset licence, attribution, privacy, and permission requirements.

12. Step-by-step: set up the project on Windows 11

Open PowerShell and change directory to the actual project folder. Based on the extraction location used in this conversation:

```
cd "C:\Users\HP\Desktop\Physics_Informed_Solar_Energy_Analytics_GitHub_Ready\Physics_Informed_Solar_Energy_Analytics"
```

Confirm the location and files:

```
Get-Location
dir
```

Create a virtual environment:

```
py -m venv .venv
```

Activate it:

```
.\.venv\Scripts\Activate.ps1
```

If PowerShell blocks activation, use this temporary setting for the current PowerShell process only, then activate again:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

Install dependencies:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If `py` is not recognised, check whether Python is installed:

```
python --version
py --version
```

If neither works, install Python 3 from the official Python website, select the option to add Python to PATH during installation, reopen PowerShell, and retry.

13. Run the analysis code before opening the notebook

Run the main analysis script from the project root:

```
python src/run_analysis.py
```

Wait for it to finish. Read the terminal output. Then inspect the updated files under `reports/` and `figures/` (if present). Do not assume the run succeeded merely because the command returned to the prompt. Look for errors and check output timestamps or contents.

Run the tests:

```
python -m pytest -q
```

The supplied project previously passed two dataset tests. Your local result may differ if you changed the files or environment. If a test fails, read the failure message and investigate it rather than deleting the test.

14. Open and run the Jupyter notebook

Install Jupyter if it is not already installed:

```
pip install notebook
```

Launch Jupyter from the project root:

```
python -m notebook
```

A browser tab should open. If it does not, copy the local URL printed in PowerShell into your browser.

In Jupyter:

1. Open the `notebooks` folder.
2. Open `project_walkthrough.ipynb`.
3. Confirm the notebook kernel uses the project's `.venv` Python environment. If it does not appear, register it:

```
pip install ipykernel
python -m ipykernel install --user --name solar-analytics --display-name "Python (Solar Analytics)"
```

4. In the notebook, select **Kernel** → **Change Kernel** → **Python (Solar Analytics)**.
5. Read the Markdown explanation at the top of each section.
6. Run cells in order from the first cell to the last using **Shift + Enter**.
7. Inspect every output, table, and plot. Do not run cells randomly before understanding their dependencies.
8. If a cell fails, read the complete error, check the file path and active kernel, and correct the cause.
9. After completing the notebook, restart the kernel and use **Run** → **Run All Cells** to check that the notebook works from a clean state.
10. Save the notebook after your own annotations and experiments.

Recommended learning sequence: first read the README, then inspect the CSV, then read `src/run_analysis.py`, run the script, inspect every report and figure, run the tests, and only then work through the notebook. The notebook is a guided explanation; the Python script is the reproducible pipeline.

15. Your personal study and demonstration checklist

Before contacting a professor or discussing the project in an interview, make sure you can:

- [ ] Explain the research problem in 60 seconds.
- [ ] State clearly that the dataset is synthetic.
- [ ] Explain every variable used by the forecast model.
- [ ] Explain the simplified PV equation and its assumptions.
- [ ] Recreate the virtual environment and install requirements.
- [ ] Run `python src/run_analysis.py` without copying someone else's output.
- [ ] Run `python -m pytest -q` and explain what the tests check.
- [ ] Open and run the Jupyter notebook from top to bottom.
- [ ] Explain the chronological split.
- [ ] Interpret MAE, RMSE, R^2, precision, recall, F1, and ROC-AUC.
- [ ] Explain why accuracy is misleading with imbalanced fault labels.
- [ ] Perform the leakage ablation excluding `performance_ratio` and `actual_power_kw`.
- [ ] Explain one limitation and one next experiment.
- [ ] Verify that your public GitHub repository contains the latest version.
- [ ] Be able to explain which parts you personally changed, tested, and understood.

Final advice: Professors are more likely to be impressed by a modest project that you can defend rigorously than by impressive-looking metrics you cannot explain. Treat this repository as the beginning of a research conversation, not as proof that the problem has been solved.